PROJECT BRIEF + MASTER PROMPT

AskMyData

AI-Powered Natural-Language Data Analytics Platform

A public web app where users ask questions about data in plain language — the system plans the analysis, queries MySQL, calculates with Python, creates charts, and explains the result with evidence-backed insights, recommendations, and warnings.

React | FastAPI | MySQL | Pandas / NumPy | Matplotlib | Gemini

Presentation-ready project overview • roadmap • architecture • security • master build prompt

1. Executive Summary

What we are actually building

CORE IDEA

Natural Language

Ask questions in normal English/Hinglish instead of writing SQL.

Deterministic Analytics

MySQL handles data retrieval; Python/Pandas handles calculations; charts come from the actual result.

AI Orchestration

Gemini interprets intent, produces a structured analysis plan, and explains real results.

Actionable Output

Every answer can include charts, insights, recommendations, and warnings.

One-line positioning: **“An AI data analyst that turns natural-language questions into safe, explainable SQL + analytics + visualizations.”**

2. Problem We Want to Solve

Current analytics workflows force users to translate business questions into technical queries

PROBLEM

- Users may understand the business question but not SQL.
- Data often lives in spreadsheets or relational tables with different schemas.
- A useful answer may require multiple steps: filtering → grouping → calculation → visualization.
- A chart alone does not explain what the result means or what action to take.

What users want to say

“Compare placement rates for students with and without internships.”

What the system should do

- Understand intent
- Generate safe analytics plan
- Query the database
- Calculate the metric
- Create the chart
- Explain the finding

3. Product Vision

From “ask a question” to “understand the data”

1. **Ask**

   User types a natural-language question.

2. **Understand**

   Gemini identifies intent, dimensions, metrics, filters and chart type.

3. **Retrieve**

   Backend builds/validates a read-only SQL query and runs it on MySQL.

4. **Analyze**

   Pandas/NumPy computes exact metrics from returned data.

5. **Visualize**

   Matplotlib/Plotly produces the appropriate chart.

6. **Explain**

   Gemini summarizes the actual results and produces insights.

7. **Act**

   Recommendations and warnings help the user decide what to investigate next.

4. End-User Experience

The core user journey we want on the public website

1. **Google Login**
2. **Choose / Upload Dataset**
3. **Ask in Natural Language**
4. **System Analyzes**
5. **See Chart + Results**
6. **Read Insights**
7. **Review Recommendations & Warnings**
8. **Ask Follow-up Question**

Follow-up conversation is optional in MVP; it becomes a major feature after the first working analytics loop.

5. Core Example: Internship vs Placement

The single example that explains the product to almost anyone

USER QUESTION

> **“What is the placement rate for students with and without internships?”**

STRUCTURED PLAN

```
{
 "group_by":"internship",
 "metric":"placement_rate",
 "chart":"bar"
}
```

GENERATED SQL (illustrative)

```
SELECT internship,
    COUNT(*) AS total_students,
    SUM(placement_status = 'Placed') AS placed
FROM students
GROUP BY internship;
```

RESULT

Internship: YES → 86%
Internship: NO → 50%

THEN Python calculates the exact metric → chart engine visualizes it → Gemini explains the result and generates recommendations + warnings.

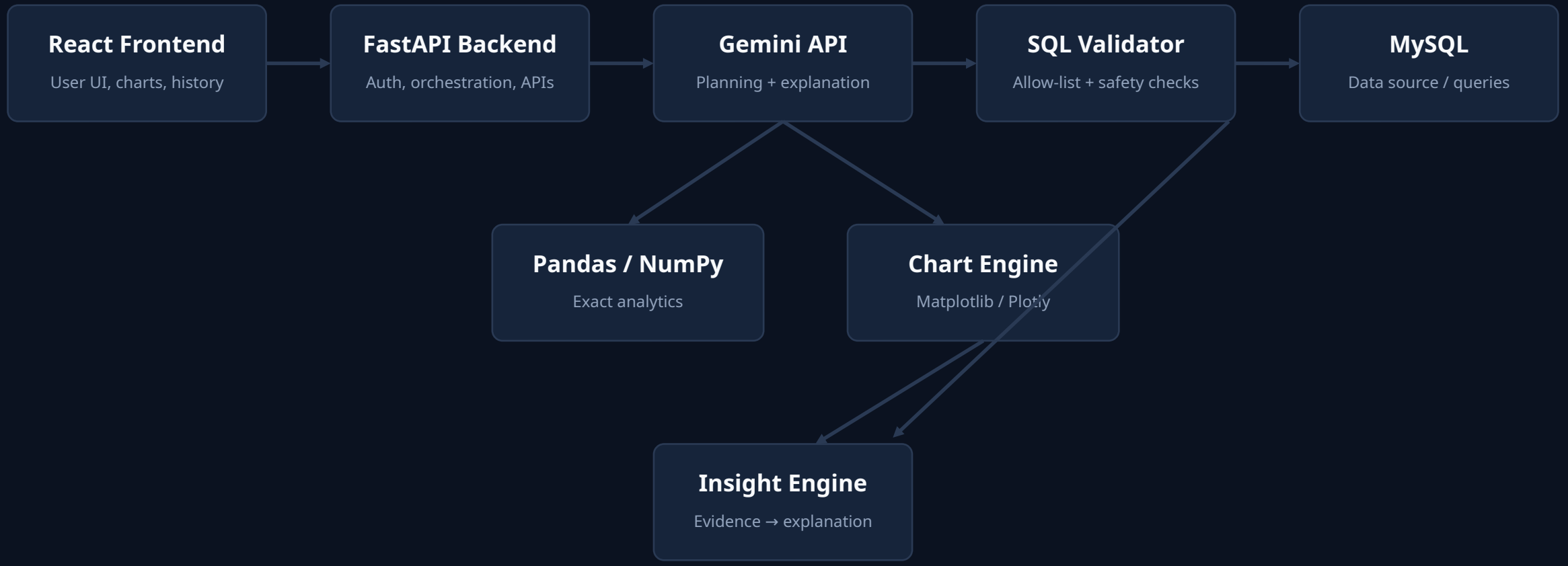
6. Main Product Modules

The application is an orchestration platform, not just a chatbot

- **Authentication** — Google OAuth; user identity and access control.
- **Dataset Manager** — Upload/select CSV, profile it, validate and isolate data.
- **Schema Registry** — Stores table/column meanings used by the AI planner.
- **AI Planner** — Natural language → structured analytics plan.
- **SQL Engine** — Safe read-only SQL generation + validation + execution.
- **Analytics Engine** — Pandas/NumPy calculations on returned data.
- **Chart Engine** — Bar, line, pie, histogram, scatter, box, etc.
- **Insight Engine** — Explain findings, recommendations and warnings.
- **History** — Store queries, plans and results for follow-up conversations.
- **Security & Limits** — Per-user access, rate limits, size limits, timeouts.

7. System Architecture

Public web application with AI orchestration and deterministic execution



Key principle: AI proposes; validated backend components execute.

8. AI Responsibilities vs Deterministic Responsibilities

Keep the model powerful but controlled

GEMINI / AI

- Understand the user’s natural-language intent
- Choose dimensions, metrics, filters and chart type
- Produce structured JSON plans
- Generate candidate read-only SQL
- Call approved backend tools/functions when enabled
- Explain actual numerical results
- Draft recommendations and warnings from structured findings

BACKEND / CODE

- Own the database connection and credentials
- Validate SQL and block writes / destructive statements
- Execute SQL and calculations
- Compute metrics deterministically
- Generate charts using known functions
- Enforce per-user access and rate limits
- Log errors, timeouts and audit information

Never let the LLM directly run arbitrary Python, shell commands, or unrestricted SQL.

9. Natural Language → Analytics Plan

The AI should return a predictable contract, not random prose

USER

“Compare average package by branch and show a bar chart.”

GEMINI → STRUCTURED OUTPUT

```
{
 "intent":"grouped_metric",
 "group_by":"branch",
 "metric":"avg_package",
 "chart":"bar"
}
```

BACKEND

- Validate fields against the schema
- Generate/validate read-only SQL
- Run query
- Pass actual result to analytics/chart engine

Why JSON/structured output?

- Predictable fields for backend validation
- Easier testing than parsing free-form text
- Works well with tool/function calling
- Makes the system easier to explain and debug

Technical reference: Google Gemini structured outputs support JSON Schema for predictable, typed responses and agentic workflows. https://ai.google.dev/gemini-api/docs/structured-output

10. SQL Safety Layer

AI-generated SQL must never become a direct database backdoor

1. Generate

Gemini proposes a query.

2. Validate

Schema, tables, columns, SQL type, row limits, timeout.

3. Execute

Only allowed read-only statements reach MySQL.

4. Observe

Log question, query template, timing, row count, errors.

Allow-list example

```
Allowed: SELECT, WITH
Blocked: DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE
```

Additional controls

- Per-user database access boundaries
- Maximum result size / pagination
- Query timeout
- Reject unknown columns/tables
- Sanitize file names and inputs

11. Python + Visualization Engine

Python is the deterministic analytics layer

Data loading	SQL result → DataFrame
Cleaning	Types, missing values, duplicates
Metrics	Rate, average, median, growth, distributions
Grouping	Group by dimensions
Statistics	Correlation / spread / outliers
Charts	Bar, line, pie, histogram, scatter, box
Export	PNG/CSV or dashboard-ready data

Design rule: AI tells the engine WHAT to calculate/show; code decides HOW to execute it.

12. Insights, Recommendations & Warnings

Turn a number into an understandable decision-support response

Answer

Exact result from the database/analytics engine.

Key insight

What pattern or difference is visible?

Recommendation

What could be investigated or improved next?

Warning

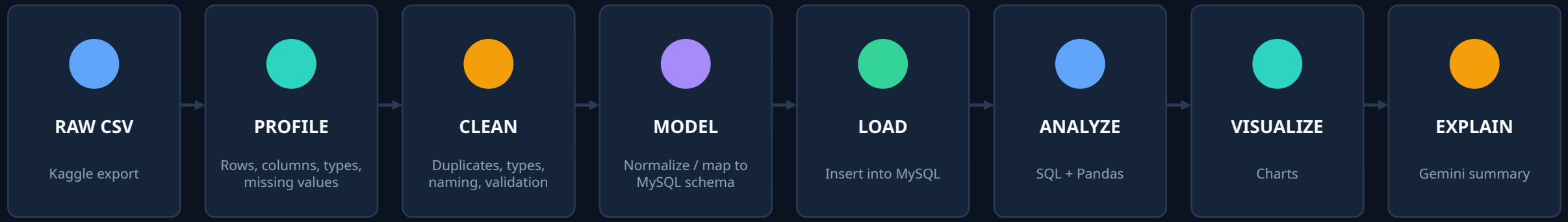
Where could the interpretation be misleading?

Example warning logic

- Small group → “interpret cautiously”
- High missingness → “metric may be unreliable”
- Correlation → do not claim causation
- Extreme outliers → consider median and distribution

13. Dataset Lifecycle

First dataset: College Placement Analytics; later the engine becomes reusable



Raw data is preserved; processed data is reproducible.

Phase 1 deliverable: inspect + clean + document the Kaggle dataset before designing MySQL.

14. Initial Database Design

The placement dataset becomes a real relational database, not just a CSV dump



Exact final schema depends on the selected Kaggle dataset. Do not invent columns before inspection.

15. Technology Stack

Keep the stack simple enough to build, but strong enough to scale the idea

Frontend	**React + Tailwind CSS**	UI, query box, dashboards, history
Backend	**Python + FastAPI**	APIs, auth, orchestration, validation
Database	**MySQL**	Relational data + read-only analytics queries
Analytics	**Pandas + NumPy**	Deterministic calculations
Charts	**Matplotlib + Plotly**	Static + interactive visuals
AI	**Gemini API**	Intent, planning, tool calling, explanations
Auth	**Google OAuth**	User identity and access
Dev / Deployment	**Git + GitHub; free-tier hosting initially**	Versioning and public MVP

16. Zero-Cost / Low-Cost Strategy

Target: build and demo the MVP with free tiers; do not assume unlimited public usage

Development	Local MySQL + Python + React + FastAPI
AI	Use Gemini free tier where available; enforce app-level rate limits
Hosting	Start with free-tier hosting for the MVP
Storage	Keep datasets small for the first public demo
Scale-up path	Move heavy workloads to paid infrastructure only after usage justifies it

Current Google docs: Gemini API has a free tier for some models, but rate limits apply and quotas vary by model/tier.

Sources: Google AI for Developers — Pricing and Rate Limits. https://ai.google.dev/gemini-api/docs/pricing • https://ai.google.dev/gemini-api/docs/rate-limits

17. Authentication & User Isolation

Google login is for identity and access—not for borrowing the user's personal AI subscription

Login flow

1. **Google Login**

   User chooses "Continue with Google".

2. **Backend Session**

   FastAPI verifies the user and creates a session.

3. **Data Isolation**

   User sees only datasets and history tied to their account.

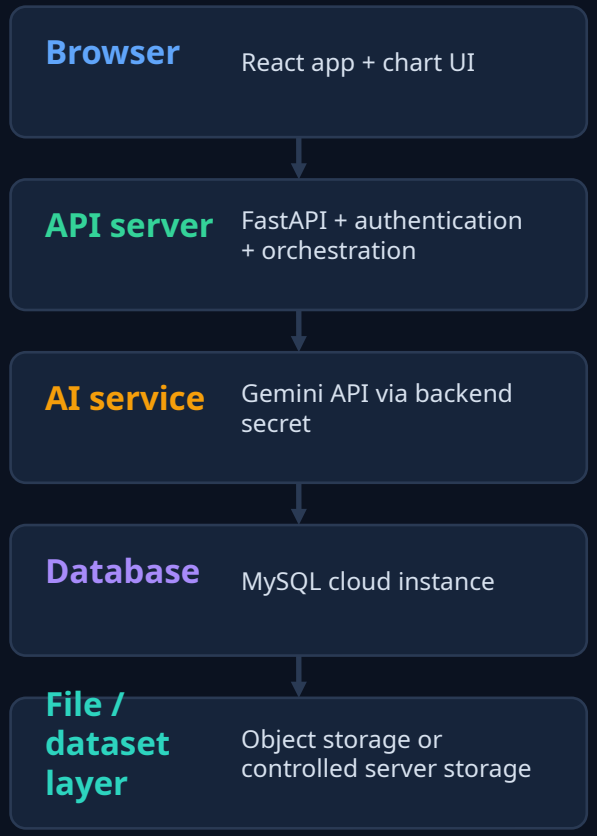
Important distinction

- Google OAuth authenticates the user.
- It does not grant access to the user's ChatGPT/Gemini subscription.
- The app uses its own backend AI configuration.
- Never ask users to paste passwords or secret API keys.

Store only what the product needs; protect access with per-user authorization.

18. Public Deployment Architecture

Development can be local; the final product must expose a secure online backend



Public request lifecycle

- User submits question
- Backend loads user/schema context
- Gemini returns structured analytics plan
- Validator checks the plan/query
- MySQL returns data
- Pandas/NumPy computes metrics
- Chart engine creates visualization
- Gemini explains actual result
- Frontend renders answer + chart + insights

19. Full Roadmap — 24 Phases

Build in a strict order so each phase leaves a working artifact

0 Planning	1 Dataset	2 DB design	3 Load data	4 SQL analytics	5 Python analytics
6 Chart engine	7 FastAPI	8 React	9 Google OAuth	10 Gemini	11 NL→SQL
12 NL→Charts	13 AI insights	14 Recommendations	15 Warnings	16 Chat history	17 Transparency
18 CSV upload	19 Multi-dataset	20 Security	21 Testing	22 Deployment	23 Polish

Rule: never start a later phase until the previous phase is testable.

20. Phase-by-Phase Deliverables

What “done” means at each major milestone

Phases 0–3	Dataset inspected → cleaned → MySQL schema → data loaded
Phases 4–6	25–40 SQL analyses → reusable Python analytics → reusable chart functions
Phases 7–9	FastAPI APIs → React UI → Google login
Phases 10–12	Gemini planner → safe NL→SQL → automatic visualization
Phases 13–15	Evidence-based insights → recommendations → warning engine
Phases 16–19	Chat history → transparent workflow → user CSV uploads → multiple datasets
Phases 20–23	Security → tests → public deployment → polished UX

21. MVP Definition

The first version should prove the full loop—not every feature

MVP MUST HAVE

- One Kaggle placement dataset
- MySQL database
- FastAPI backend
- React query interface
- Gemini structured planning
- Safe read-only SQL execution
- Pandas metric calculation
- Matplotlib/Plotly chart
- AI explanation
- Basic warning handling

NOT IN FIRST DEMO

- Multiple datasets
- Complex conversational memory
- Advanced recommendation engine
- Enterprise permissions
- Heavy scaling
- Fancy animations
- Paid AI infrastructure

Goal: one question → correct result → useful chart → understandable explanation.

22. Phase 1 — Exact Work for Tomorrow

This is the first task to give GitHub Copilot

Project structure

data/raw • data/processed • data/sample • scripts • notebooks • docs

Dataset inspector

Rows, columns, types, head/tail, missing values, duplicates, unique counts

Cleaning script

Standardize names, remove exact duplicates, handle obvious issues, log changes

Data dictionary

Column name, type, example, missing count, meaning (unknown = needs confirmation)

README

How to run inspection/cleaning; project scope; current phase

Dependencies

Only what Phase 1 needs: pandas + numpy (plus standard library)

No Gemini • No MySQL • No React • No Auth in Phase 1.

23. Information We Need From the Team

Use this slide to collect decisions before implementation expands

Dataset — Which Kaggle dataset? CSV size? License/usage constraints?

Schema — Should we normalize into Students / Internships / Placements / Companies, or keep a simpler schema for MVP?

Users — Expected number of users for the first demo? Internal only or public?

Privacy — Will users upload private data? If yes, what retention/deletion policy is required?

AI — Use Gemini free tier for MVP? What app-level request limit should we enforce?

Charts — Which chart types are mandatory in v1?

Deployment — Preferred free-tier hosting providers, domain, or institutional server?

Evaluation — Which 20–30 natural-language questions define “working correctly”?

24. Future Expansion

What this can become after the first working version

Dataset auto-discovery — Upload CSV → infer schema → generate metadata automatically.

Multi-dataset projects — Join multiple datasets when relationships are declared/verified.

Conversational analytics — Follow-up questions that reuse context and filters.

Scheduled reports — Daily/weekly automated summaries.

Exportable reports — PDF/CSV/PNG export and shareable analysis snapshots.

Advanced statistics — Confidence intervals, A/B tests, forecasting, anomaly detection.

Tool/function calling — Gemini selects approved analytics tools rather than generating arbitrary code.

Observability — Usage metrics, latency, query performance, AI error rates.

Gemini function calling is supported for tool-driven workflows; structured outputs can constrain responses to a schema. https://ai.google.dev/gemini-api/docs/function-calling

25. Risks & How We Handle Them

The project should be judged on correctness, not only on how impressive the demo looks

Hallucinated SQL	Schema validation + allow-list + query tests
Wrong insights	Only summarize calculated results; never invent numbers
Causal overclaim	Use warnings for observational/correlation findings
Large / slow queries	Pagination, row limits, timeouts
Private uploads	Clear policy, isolation, deletion controls
Free-tier exhaustion	Per-user request limits + lightweight prompts
Model changes	Keep AI behind an abstraction layer
User ambiguity	Ask clarification or show assumptions before executing risky analysis

26. Success Criteria

How we will know the platform is actually working

- ✓ A user can sign in securely.
- ✓ A validated CSV can be profiled and stored.
- ✓ The same dataset can be queried through MySQL.
- ✓ At least 20 natural-language questions are mapped correctly.
- ✓ AI-generated SQL passes validation and returns correct results.
- ✓ Charts are generated automatically from structured analysis plans.
- ✓ AI explanations match the actual returned numbers.
- ✓ Recommendations and warnings are tied to measurable findings.
- ✓ Each user can access only their own data.
- ✓ The MVP can run online within agreed free-tier limits.

27. Master Build Prompt — Role & Project Context

This is the prompt to give an AI coding agent / GitHub Copilot

```
You are a senior full-stack + data-engineering assistant helping build "AskMyData", an AI-powered public data analytics platform.

FINAL PRODUCT
A React web app where users can sign in with Google, select/upload a dataset, ask questions in natural language, and receive:
• validated SQL-based answers
• Pandas/NumPy calculations
• Matplotlib/Plotly visualizations
• concise insights
• evidence-backed recommendations
• warnings/limitations
• optionally the generated SQL and analysis plan

STACK
Frontend: React + Tailwind CSS
Backend: Python + FastAPI
Database: MySQL
Analytics: Pandas + NumPy
Charts: Matplotlib + Plotly
AI: Gemini API (free tier for MVP where available)
Auth: Google OAuth

ARCHITECTURE PRINCIPLE
Gemini plans and explains; validated backend code executes. Never allow arbitrary LLM-generated Python, shell commands, or unrestricted SQL.
```

Important: build phase-by-phase. Do not implement later phases automatically.

28. Master Build Prompt — Engineering Rules

Use these constraints for every future coding phase

```
ENGINEERING RULES
1. Inspect the current workspace before changing files.
2. Do not delete existing work without explicit instruction.
3. Keep modules small and testable.
4. Use environment variables for secrets.
5. Use type hints and clear error handling.
6. Use pathlib / safe paths; never hardcode machine-specific absolute paths.
7. Preserve raw datasets; store processed versions separately.
8. Validate all user inputs.
9. Treat SQL as read-only unless a future phase explicitly adds controlled write operations.
10. Apply per-user authorization to datasets and history.
11. Do not invent schema columns; derive them from the real dataset.
12. Prefer structured JSON plans for AI outputs.
13. Unit-test the deterministic analytics engine independently of the AI.
14. Log query latency, errors and result sizes without exposing secrets.
15. Keep the AI provider behind a small service interface so the model can be changed later.
16. After each phase, report files changed, how to run, how to test, and what remains.
```

29. Master Build Prompt — Phase Control

This prevents an AI coding agent from trying to build everything at once

```
PHASE ORDER
0 Planning
1 Dataset inspection + cleaning
2 MySQL schema design
3 Data import
4 Manual SQL analytics library
5 Python analytics engine
6 Chart engine
7 FastAPI backend
8 React frontend
9 Google OAuth
10 Gemini integration
11 Natural language → SQL
12 Natural language → chart/analysis plan
13 AI insights
14 Recommendations
15 Warnings
16 Chat history
17 Transparent SQL/analysis display
18 User CSV upload
19 Multiple datasets
20 Security hardening
21 Testing
22 Deployment
23 UX polish

CURRENT RULE
Only work on the phase explicitly requested. If a phase requires information from me, stop and ask concise questions rather than inventing requirements.
```

30. Master Build Prompt — Phase 1 Exact Task

The first implementation prompt to paste into GitHub Copilot

```
PHASE 1: DATASET UNDERSTANDING & CLEANING FOUNDATION

• Inspect the existing workspace first.
• Create: data/raw, data/processed, data/sample, scripts, notebooks, docs.
• Create scripts/inspect_dataset.py that accepts a CSV path and prints rows, columns, types, head/tail, missing values, duplicates, unique counts and numeric statistics.
• Create scripts/clean_dataset.py that standardizes snake_case names, removes exact duplicates, handles obvious issues carefully, logs every change, and writes data/processed/placement_cleaned.csv.
• Create docs/data_dictionary.md with column name, meaning, type, example and missing-count fields. Never invent unclear meanings; mark them "Needs confirmation".
• Create/update README.md with Phase 1 instructions.
• requirements.txt should contain only dependencies required for this phase (pandas, numpy, standard library).
• Do not implement Gemini, MySQL, React, FastAPI, Google OAuth, AI-to-SQL, recommendations, deployment or multi-user features yet.
• Finish by reporting: directory tree, files changed, install command, run commands, validation performed, and what information is needed before Phase 2.
```

31. Suggested Questions to Ask Others Tomorrow

Use these to collect requirements before the team commits to implementation

1. Do we already have a Kaggle dataset in mind?
2. What exact questions should the website answer first?
3. Do we want one-table MVP or normalized multi-table schema?
4. Will users upload personal/confidential datasets?
5. How many users should the public demo support?
6. Which charts are mandatory?
7. Do we want export as PNG/PDF/CSV in v1?
8. Where should the online MVP be hosted?
9. What Gemini free-tier limit should we enforce per user?
10. Who owns testing / UI / backend / database / deployment?

Best outcome from tomorrow's discussion: freeze the Phase-1 dataset + MVP question list + deployment constraints.

32. Immediate Next Steps

What we do next, in order

Today	Choose the Kaggle dataset and place the CSV in data/raw/.
Step 1	Run Phase-1 inspector + cleaning scripts.
Step 2	Review the data dictionary and confirm ambiguous fields.
Step 3	Design the MySQL schema from the real dataset.
Step 4	Load the database and validate row counts / relationships.
Step 5	Build the first 20–30 analytics questions.
Step 6	Only then integrate FastAPI, React and Gemini.

First milestone: a clean, trustworthy MySQL database that can answer real placement questions.

33. Current Technical References

Official references used for the current Gemini-related design decisions

Gemini API Pricing	https://ai.google.dev/gemini-api/docs/pricing	Free tier exists for selected models; paid tiers and data-use terms differ.
Gemini API Rate Limits	https://ai.google.dev/gemini-api/docs/rate-limits	Usage is subject to project-level rate limits and model/tier constraints.
Structured Outputs	https://ai.google.dev/gemini-api/docs/structured-output	JSON-schema-constrained responses fit the analytics-plan contract.
Function Calling	https://ai.google.dev/gemini-api/docs/function-calling	Supports tool-oriented workflows where the model requests approved backend functions.
Gemini Quickstart	https://ai.google.dev/gemini-api/docs/get-started	Official SDK/API setup and structured-output examples.

Accessed for this deck: 25 Sep 2026. Re-check limits and pricing immediately before launch because provider terms can change.

ASKMYDATA

Build the first working loop.

Dataset → MySQL → SQL → Pandas → Chart → Gemini → Insight

Tomorrow’s concrete outcome

Freeze the dataset + agree on the first analytics questions + start Phase 1.

End of brief